CPIA · DOCUMENTACIÓN TÉCNICA DE CÓDIGO

Explicación Línea a Línea del Script

Desglose completo y justificación de `problemas/5_Materials_Uriel/materiales_uriel.py`

Estudiante: Uriel | **Algoritmo:** DBSCAN (ε = 0.65, min_samples = 3) | **Archivo explicado:** `materiales_uriel.py`

1. Bloque de Importaciones (Líneas 13 – 16)

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

- **Línea 13 (`from pathlib import Path`):** Importa la clase `Path` para gestionar rutas de archivos de forma independiente del sistema operativo (evita fallos de barras inclinadas `/` o contrabarras `\` entre Windows, Mac y Linux).
- **Línea 14 (`import numpy as np`):** Carga NumPy para operaciones matriciales vectorizadas de alta velocidad (matrices de distancias euclídeas, medias y desviaciones típicas).
- **Línea 15 (`import pandas as pd`):** Carga Pandas para leer el archivo CSV en un DataFrame estructurado y generar tablas de contingencia cruzada.
- **Línea 16 (`import matplotlib.pyplot as plt`):** Carga el motor gráfico para generar la visualización 2D de los clusters y guardarla en formato PNG.

2. Carga del Dataset y Rutas Relativas (Líneas 21 – 28)

```
DATOS_DIR = Path(__file__).resolve().parent.parent / "datos"
CSV_PATH = DATOS_DIR / "materiales_electricos.csv"

if not CSV_PATH.exists():
    raise FileNotFoundError(f"No se encontró el archivo de datos en: {CSV_PATH}")

df = pd.read_csv(CSV_PATH)
print(f"Dataset cargado con éxito ({len(df)} materiales).")
```

- **Línea 21:** `__file__` es la ruta de este script (en `problemas/`). `.resolve()` obtiene la ruta absoluta completa; `.parent` sube a `problemas/` y el segundo `.parent` sube a la raíz `CPIA-Expo/` . Finalmente, `/ "datos"` apunta con exactitud a la carpeta de datos sin importar desde qué directorio abra la terminal el usuario.
- **Líneas 24 – 25:** Comprobación de seguridad: si el archivo no existe, lanza un error explicativo en lugar de colgarse más adelante.
- **Línea 27:** Lee el fichero CSV con los 57 materiales y crea el DataFrame `df` .

3. Selección de Características y Estandarización Z-Score (Líneas 35 – 41)

```
features = ["log_rho_300K", "alpha_coef"]
X_raw = df[features].values

media = np.mean(X_raw, axis=0)
desv = np.std(X_raw, axis=0)
X_scaled = (X_raw - media) / desv
```

- **Líneas 35 – 36:** Selecciona las dos columnas físicas del espacio bidimensional: la resistividad logarítmica (rango ≈ 31 unidades) y el coeficiente térmico α (rango ≈ 0.08 unidades). Extrae la matriz NumPy `X_raw` de dimensiones `(57, 2)`.
- **Líneas 39 – 40:** Calcula para cada columna su media μ y su desviación estándar σ.
- **Línea 41:** Aplica la fórmula de estandarización **Z = (X − μ) / σ**. Esto es imprescindible porque la distancia euclídea de DBSCAN mide la separación en línea recta. Sin este paso, la resistividad (con escala de 31) aplastaría a α (escala de 0.08), haciendo que el algoritmo fuese ciego a la respuesta térmica.

4. Aplicación de DBSCAN Paso a Paso (Líneas 46 – 89)

```
EPS = 0.65
MIN_SAMPLES = 3

def ejecutar_dbscan(X, eps=0.65, min_samples=3):
    try:
        from sklearn.cluster import DBSCAN as SklearnDBSCAN
        return SklearnDBSCAN(eps=eps, min_samples=min_samples).fit_predict(X)
    except Exception:
        N = len(X)
        etiquetas = np.full(N, -1, dtype=int)
        visitados = np.zeros(N, dtype=bool)
        cluster_actual = 0

        D = np.linalg.norm(X[:, None, :] - X[None, :, :], axis=-1)

        for i in range(N):
            if visitados[i]: continue
            visitados[i] = True

            vecinos = np.where(D[i] <= eps)[0]
            if len(vecinos) < min_samples:
                etiquetas[i] = -1
            else:
                etiquetas[i] = cluster_actual
                cola = list(vecinos[vecinos != i])
                while cola:
                    j = cola.pop(0)
                    if not visitados[j]:
                        visitados[j] = True
                        j_vecinos = np.where(D[j] <= eps)[0]
                        if len(j_vecinos) >= min_samples:
                            for v in j_vecinos:
                                if v not in cola and etiquetas[v] == -1:
                                    cola.append(v)
                    if etiquetas[j] == -1:
                        etiquetas[j] = cluster_actual
                cluster_actual += 1
        return etiquetas

df["cluster"] = ejecutar_dbscan(X_scaled, eps=EPS, min_samples=MIN_SAMPLES)
```

- **Líneas 46 – 47:** Fija los dos hiperparámetros: radio de búsqueda ε = 0.65 y mínimo de muestras para densidad `min_samples = 3`.
- **Línea 62 (Matriz de distancias vectorizada):** Con `X[:, None, :] - X[None, :, :]` aprovecha el *broadcasting* de NumPy para calcular las distancias euclídeas de todas las parejas de materiales en una matriz `D` de 57 × 57 de una sola vez sin bucles lentos.
- **Líneas 69 – 71:** Para el punto `i`, busca quiénes están a distancia ≤ ε. Si tiene menos de 3 vecinos, lo marca provisionalmente como ruido (`-1`).
- **Líneas 73 – 86 (Expansión BFS del cluster):** Si el punto es Núcleo, le asigna `cluster_actual` y añade a sus vecinos a una lista `cola`. Para cada vecino `j`, si también es Núcleo, expande el cluster agregando sus propios vecinos a la cola. Cuando la cola se vacía, incrementa `cluster_actual += 1`.
- **Línea 89:** Guarda las etiquetas finales (0, 1, 2) en una nueva columna `df["cluster"]`.

5. Evaluación con Silhouette Score (Líneas 94 – 120)

```
def calcular_silhouette(X, labels):
    mask = labels != -1
    if len(set(labels[mask])) <= 1: return None
    D = np.linalg.norm(X[:, None, :] - X[None, :, :], axis=-1)
    sils = []
    for i in range(len(X)):
        if labels[i] == -1: continue
        c_i = labels[i]
        same_c = np.where((labels == c_i) & (np.arange(len(X)) != i))[0]
        a_i = np.mean(D[i, same_c]) if len(same_c) > 0 else 0
        other_clusters = set(labels) - {-1, c_i}
        b_i = min(np.mean(D[i, labels == c_o]) for c_o in other_clusters)
        sils.append((b_i - a_i) / max(a_i, b_i))
    return float(np.mean(sils))
```

- **Línea 95:** `mask = labels != -1` excluye los puntos de ruido para no falsear la silueta interna de los clusters densos.
- **Línea 106:** Para cada material `i` calcula `a_i` (distancia media a los materiales de su mismo cluster → *cohesión*).
- **Línea 108:** Calcula `b_i` (distancia media al cluster vecino más cercano → *separación*).
- **Línea 109:** Aplica la fórmula exacta `s = (b - a) / max(a, b)`. El promedio resultante da **0.7858**, certificando una estructura de clusters excelente.

6. Métricas y Matriz de Contingencia (Líneas 124 – 144)

```
clusters_encontrados = sorted(set(df["cluster"]) - {-1})
n_clusters = len(clusters_encontrados)
n_ruido = int(np.sum(df["cluster"] == -1))

print(f"Clusters descubiertos por densidad : {n_clusters}")
print(f"Silhouette Score (núcleos densos) : {score_sil:.4f}")

print("\nMatriz de contingencia (Clase real vs. Cluster DBSCAN):")
print(pd.crosstab(df["clase_real"], df["cluster"], margins=True))
```

- **Líneas 124 – 126:** Cuenta cuántos clusters reales se descubrieron (excluyendo el `-1` de ruido).
- **Línea 137:** `pd.crosstab()` cruza la columna `clase_real` contra `cluster`. Demuestra de forma objetiva que el Cluster 0 agrupó a los 35 metales (100%), el Cluster 1 agrupó a los 10 aislantes (100%) y el Cluster 2 agrupó a los semiconductores tecnológicos.

7. Representación Gráfica 2D con Matplotlib (Líneas 148 – 219)

```
plt.figure(figsize=(10, 6.5))

paleta_colores = {
    0: ("#d97706", "Cluster 0 (Conductores)"),
    1: ("#2563eb", "Cluster 1 (Aislantes)"),
    2: ("#059669", "Cluster 2 (Semiconductores)"),
}

for cl in clusters_encontrados:
    sub = df[df["cluster"] == cl]
    color, nombre_cl = paleta_colores.get(cl, ("#6b7280", f"Cluster {cl}"))
    plt.scatter(sub["log_rho_300K"], sub["alpha_coef"], c=color, s=70, alpha=0.85, ...)

plt.axhline(0, color="gray", linestyle="--", linewidth=1, alpha=0.7, label=r"Frontera $\alpha = 0$")

# Anotar nombres de materiales destacados en el gráfico
materiales_destacados = ["copper", "silver", "silicon", "germanium", "gaas", "glass", "ptfe", "quartz"]
for _, r in df.iterrows():
    if any(k in r["material"].lower() for k in materiales_destacados):
        plt.annotate(r["material"].split("(")[0].strip(), (r["log_rho_300K"], r["alpha_coef"]), ...)

FIGURA_PATH = Path(__file__).resolve().parent / "clusters_materiales_dbscan.png"
plt.savefig(FIGURA_PATH, dpi=300)
plt.show()
```

- **Línea 151:** Define una paleta de colores semántica: ámbar/dorado para metales, verde para semiconductores y azul para aislantes.
- **Línea 164:** Dibuja los puntos de cada cluster con borde negro fino y transparencia para visualizar solapamientos.
- **Línea 174:** Si hubiera puntos de ruido (-1), los dibujaría con aspas `x` de color rojo fuerte.
- **Línea 186:** Traza la recta horizontal discontinua en $\alpha = 0$, que representa la frontera física fundamental de separación entre metales ($\alpha > 0$) y semiconductores ($\alpha < 0$).
- **Líneas 189 – 202:** Localiza materiales emblemáticos (Cobre, Silicio, Vidrio, Teflón, etc.) y les coloca una etiqueta de texto flotante ligeramente desplazada (`xytext=(6, 4)`) para que la gráfica sea autoexplicativa en las diapositivas.
- **Línea 213:** Guarda la figura con calidad de publicación (300 DPI) en `problemas/clusters_materiales_dbscan.png` .